CSC429

COMPUTER SECURITY

# BUILDING SECURE WEB APPLICATION

GROUP #5

King Saud University

College of Computer and Information Science

Computer Science Department

# CSC429 Computer Security

## 2nd Semester 1447

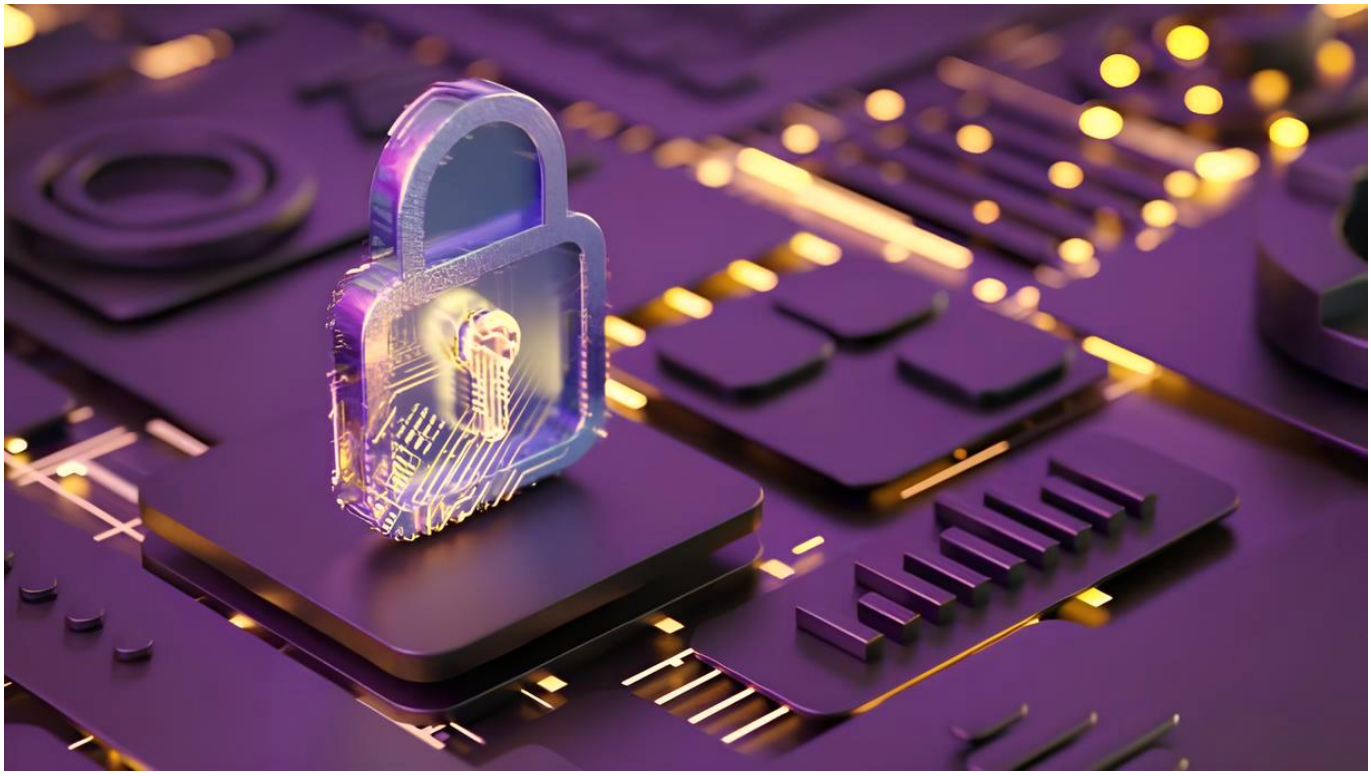
| | |
|---|---|
| **Assignment Title** | **Building a Secure Web Application - Detection and Mitigation of Security Vulnerabilities** |
| **Link GitHub** | **https://github.com/DemoTuna/Csc429-project** |
| **Date of Submission** | **8th MAY 2026** |
| | |

| **Group Number** | **5** |
|---|---|

**Group Members**

| # | Name | ID |
|---|---|---|
| 1 | Shmookh Almoliafai | 444201101 |
| 2 | Dimah Althunayan | 444201003 |
| 3 | Raghad Baselm | 444200172 |
| 4 | Jood Alotaibi | 444925028 |

# Table of Contents



# INTRODUCTION

In today's digital environment, web application security has become a critical requirement rather than an optional feature. Many applications are vulnerable to common attacks due to improper handling of user input, weak authentication mechanisms, and insufficient access control. These vulnerabilities can lead to serious risks such as data breaches, unauthorized access, and system misuse.

This project presents the development and enhancement of a secure web application that includes user registration, login functionality, and an admin dashboard. The main objective is to identify common security vulnerabilities within the application and implement effective solutions to mitigate them.

Several key vulnerabilities were analyzed, including SQL Injection, weak password storage, Cross-Site Scripting (XSS), lack of proper access control, and insufficient data protection. To address these issues, appropriate security mechanisms were applied, such as input validation, role-based access control, and secure handling of user data.

The final system ensures that sensitive operations are restricted to authorized users only, particularly within the admin panel, and demonstrates how fundamental security practices can significantly improve the overall safety and reliability of a web application.

# IDENTIFIED VULNERABILITIES

## A. SQL INJECTION

SQL Injection is a critical web security vulnerability that occurs when user input is directly incorporated into SQL queries without proper validation or sanitization. This allows attackers to manipulate the structure of the query and potentially bypass authentication or access sensitive data.

In this application, this vulnerability exists in the login functionality. The system constructs the SQL query by directly concatenating user provided inputs (username and password) into the query string, without any form of input sanitization.

The following code demonstrates the vulnerable implementation:

```
const query = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" +
password + "'";

const rows = await db.query(sql.__dangerous__rawValue(query));
```

This approach is insecure because the database cannot distinguish between legitimate input and malicious SQL code. As a result, an attacker can inject SQL commands into the input fields.

For example, entering the following input in the username field:

```
' OR 1=1 --
```

modifies the original query logic and instead of checking valid credentials, the query becomes:

```
SELECT * FROM users
WHERE username = '' OR 1=1 --' AND password = '';
```

Since the condition OR 1=1 is always true, the database returns a valid user record regardless of the password. The -- symbol comments out the remaining part of the query, effectively bypassing authentication.

As shown in Figure 1 and Figure 2, This attack successfully allows unauthorized access to the system, granting the attacker access to the admin dashboard without valid credentials. This demonstrates a complete authentication bypass.

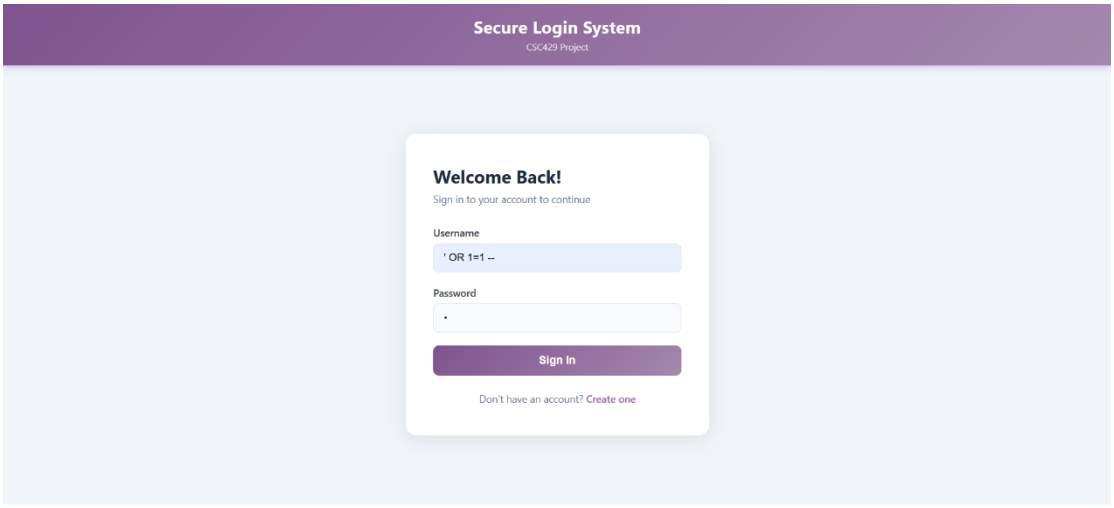



*Figure 1: SQL Injection attempt on the login page*

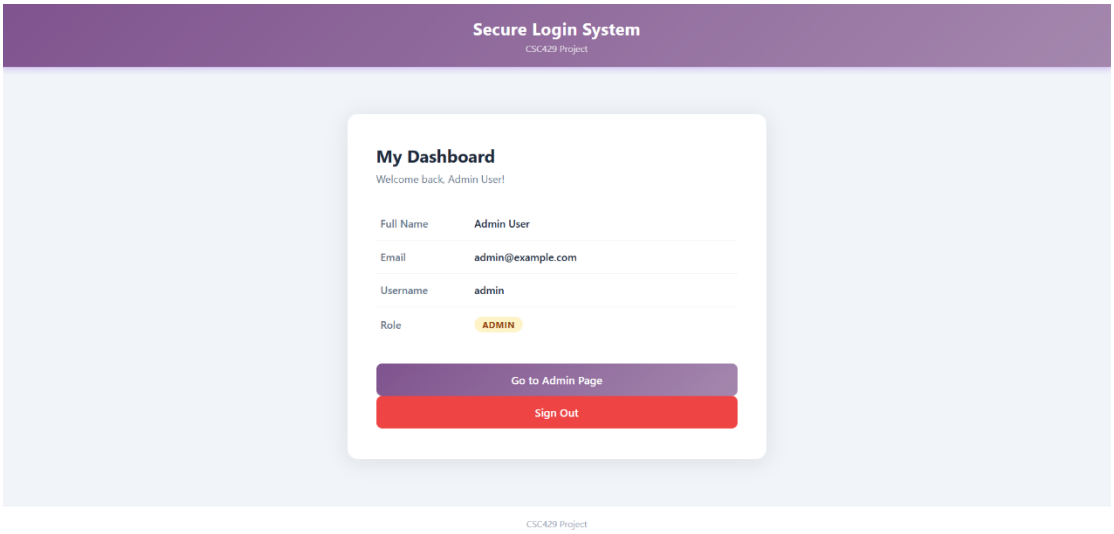



*Figure 2: Unauthorized access to the admin dashboard*

This occurs because the injected condition (OR 1=1) causes the query to return all records from the users table. Since the application selects the first returned record (rows[0]), it logs the attacker in as the first user in the database. In this case, the first record corresponds to the admin account, which results in the attacker being granted administrative access.

The impact of this vulnerability is severe, as it can lead to unauthorized access, privilege escalation, and exposure of sensitive user data. If exploited further, it could allow attackers to retrieve, modify, or delete database contents.

## B. WEAK PASSWORD STORAGE

The application was found to store user passwords in plain text within the database, which represents a critical security vulnerability. As shown in Figure 3 The password column contains readable values such as (admin123) and (Aa12344321), indicating that no hashing mechanism was applied during user registration.

| | id | full_name | email | username | password | role |
|---|---|---|---|---|---|---|
| 1 | 1 | Admin User | admin@example.com | admin | admin123 | admin |
| 2 | 2 | Test User | test@test.com | testuser | testpass | user |
| 3 | 3 | Demo Althunayan | 444201003@student.ksu.edu.sa | demotuna | Aa12344321 | user |
| 4 | 4 | Shmookh Almoliafai | shmookh@gmail.com | shmookh_almoliafai | 1234567 | user |
| 5 | 5 | Raghad Baselm | raghad@gmail.com | raghad_baselm | 12345678 | user |
| 6 | 6 | Jood Alotaibi | Jood@gmail.com | jood_alotaibi | 123456789 | user |

*Figure 3: User passwords stored in plain text in the database, demonstrating a lack of hashing or encryption*

This issue occurs because user passwords are directly stored in the database without any transformation or protection. During the registration process, the password provided by the user is inserted into the database as-is, without applying a hashing algorithm such as bcrypt.

The following code demonstrates how user passwords were stored directly in the database without any hashing:

```
await db.query(sql`
INSERT INTO users (full_name, email, username, password, role) VALUES (${full_name}, ${email},
${username}, ${password}, 'user')
`);
```

Storing passwords in plain text poses a severe security risk. If the database is compromised, an attacker can immediately view all user passwords without needing to perform any additional effort. This can lead to account takeovers, especially if users reuse passwords across multiple platforms. Furthermore, it violates fundamental security best practices, which require that passwords should never be stored in a readable format.

This vulnerability significantly impacts the confidentiality of user data and increases the risk of unauthorized access to user accounts. It also exposes the system to large-scale credential leaks in the event of a data breach.

## C. CROSS-SITE SCRIPTING (XSS)

Cross-Site Scripting is a vulnerability allowing attackers to inject malicious code into websites to run on a user's browser. This attack compromises the confidentiality and integrity of data transfers between the website and clients.

In this application, *Stored XSS (persistent XSS)* is found in the comment section, where the attacker can add a comment that contains an HTML tag or JavaScript code, and this comment is then stored in the comments database and executed whenever these comments get retrieved and appear in the comment section.

Furthermore, the application was vulnerable to XSS attacks as it was lacking Insufficient input validation and sanitization before storing, and a lack of output encoding for characters like <, >, ", ', as the browser may misinterpret text as html code and allow malicious attacks against the site.

The following code segment from the server-side shows how the comment gets stored in its raw form:

```
const { comment } = req.body;
const username = req.session.user.username;

await db.query(sql`
    INSERT INTO comments (username, comment)
    VALUES (${username}, ${comment})
`);
```

Another vulnerability on the HTML side caused unencoded characters to be misinterpreted as code:

```
comments.forEach(function (row) {
html += '<table class="info-table" style="margin-bottom:10px;"><tbody>';
html +='<tr><td style="width:120px;"><strong>' + row.username + '</strong></td>';
html += '<td>' + row.comment + '</td>';
html += '</tr></tbody></table>'; });
```

The following Figures 4 and 5 show an example of how a stored XSS attack can happen in the application.

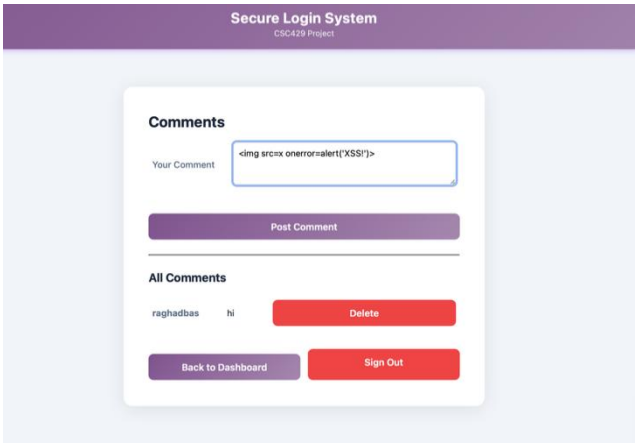



*Figure 4: The comment section before inserting the XSS attack comment.*

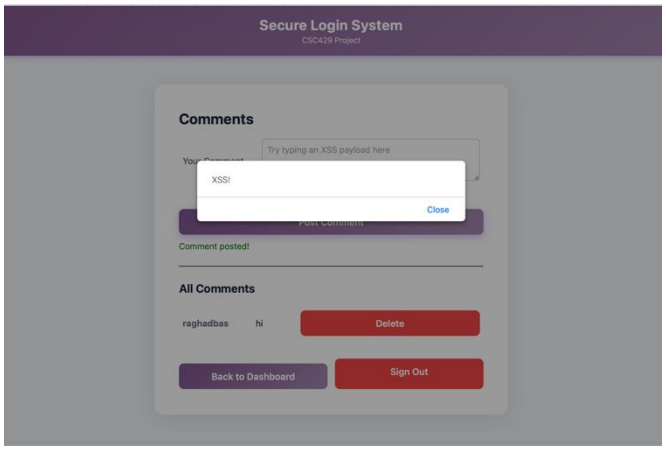



*Figure 5: The comment section after the XSS attack comment is stored and retrieved in the page.*

## D. ACCESS CONTROL

Access Control is a critical security mechanism that ensures users can only access resources and functionalities that match their assigned roles. When access control is not properly enforced, it leads to a vulnerability known as *Broken Access Control*, where users can perform actions beyond their authorized permissions.

In this application, the system initially lacked proper authorization checks for the admin page (/admin). Although the system supported different roles such as “admin” and “user”, it did not enforce these roles when accessing protected routes.

The application only verified whether the user was authenticated (logged in), but did not verify whether the user had the appropriate privileges to access administrative functionality. As a result, any authenticated user could manually access the admin page by directly entering the URL in the browser.

The following scenario demonstrates vulnerability:

1. A normal user logs into the system using valid credentials.
2. The user manually enters the URL /admin in the browser.
3. The system grants access to the admin page without verifying the user's role.

As shown in Figure 6 and Figure 7 A normal user can access the admin dashboard without administrative privileges, demonstrating unauthorized access to restricted functionality.

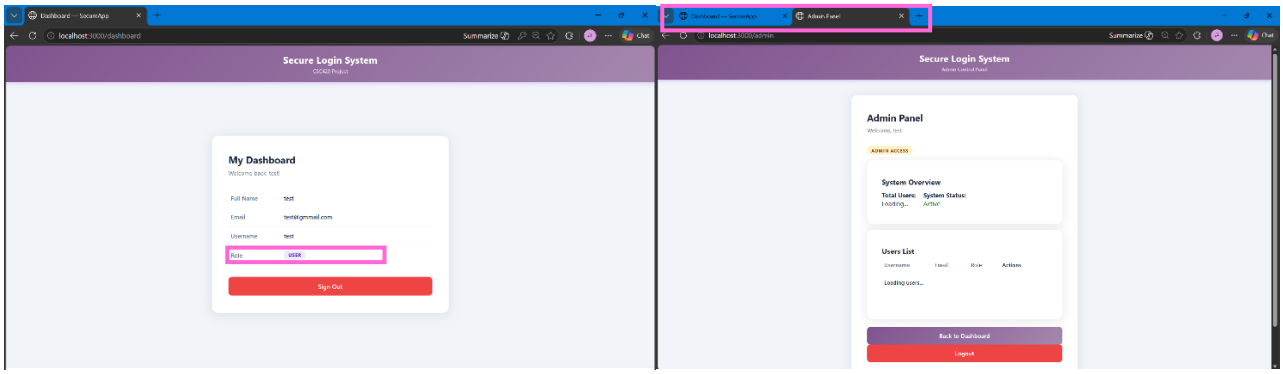



*Figure 6: Normal user attempting to access the admin page by manually entering the URL*

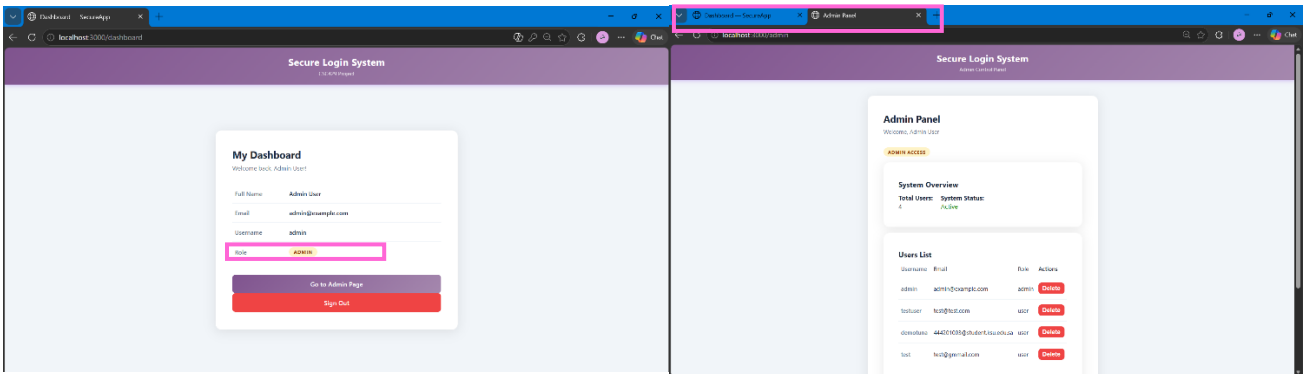



*Figure 7: Unauthorized access to the admin dashboard without role verification*

This issue occurs because the application does not enforce role-based authorization checks on the server side. Unlike authentication, which verifies user identity, authorization ensures that users can only access resources they are permitted to use. In this case, the absence of role validation allows unauthorized users to access sensitive features.

The impact of this vulnerability is significant, as it allows privilege escalation and unauthorized access to administrative functionality. Attackers could potentially exploit this flaw to view sensitive data or perform restricted actions within the system.

## E. LACK OF ENCRYPTION

The application initially lacked proper encryption mechanisms for protecting sensitive data in transmission and storage. Communication between the client and server was conducted over HTTP, meaning that all data was transmitted in plaintext. This exposed transmissions to interception attacks such as Man-In-The-Middle (MITM), where an attacker can spy on or modify sensitive information, such as login credentials.

The following code sets up the initial HTTP server without securing transmission:

```
app.listen(PORT, () => {
            console.log(`Server running at : http://localhost:${PORT}`);
});
```

In addition, the session secret used to maintain authenticated sessions for logged-in users was hardcoded in the server-side code. This practice is insecure because if the source code is exposed, attackers can access the secret key and forge valid session cookies, leading to session hijacking and unauthorized access.

The following code snippet shows how the session secret was used before mitigation:

```
app.use(session({
    secret: 'csc429-session-secret-key', // hardcoded session secret
    resave: false,
    saveUninitialized: false,
    cookie: {
        maxAge: 1000 * 60 * 60
    }
}));
```

# MITIGATION TECHNIQUES

## A. SQL INJECTION

To address the SQL injection vulnerability, the login and registration queries were modified to use parameterized queries instead of direct string concatenation. In the vulnerable implementation, user inputs were directly embedded into the SQL query string, allowing attackers to inject malicious SQL code. To prevent this, the application was updated to separate SQL logic from user input by using safe parameter binding.

The following code shows the secure implementation:

```
const rows = await db.query(sql`
            SELECT * FROM users
            WHERE username = ${username} AND password = ${password} `);
```

In this approach, the user inputs (username and password) are treated strictly as data rather than executable SQL code. The database automatically sanitizes these values, ensuring that any malicious input is not interpreted as part of the query structure.

As a result, attempts to inject SQL commands (' OR 1=1 --) no longer alter the behavior of the query and are instead processed as normal input values. This prevents attackers from bypassing authentication.

As shown in Figure 8 When the same SQL injection input is used after applying parameterized queries, the system correctly rejects the login attempt and displays an “Invalid username or password” message instead of granting access. This confirms that the implemented solution successfully mitigates the SQL injection vulnerability.

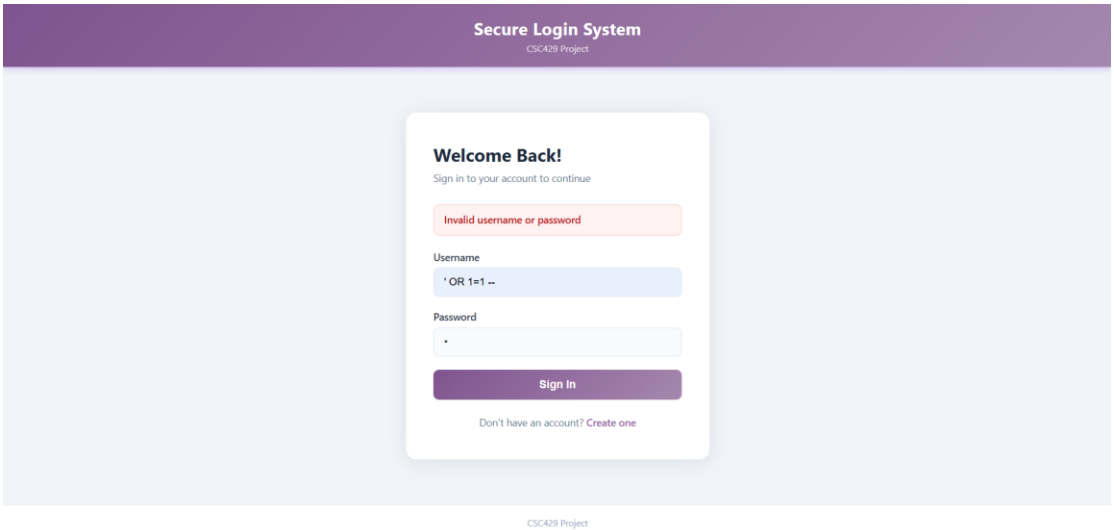



*Figure 8: SQL Injection attempt after mitigation*

Additionally, the use of __dangerous__rawValue() was completely removed, as it bypasses built-in protections and allows execution of unsafe raw SQL queries. Eliminating this function ensures that all database operations benefit from the security mechanisms provided by the database library.

Overall, this mitigation significantly improves the security of the application by preventing unauthorized access and protecting the integrity of the database.

## B. WEAK PASSWORD STORAGE

To address the weak password storage vulnerability, a secure hashing mechanism was implemented using the bcrypt library. Instead of storing passwords in plain text, each password is transformed into a hashed value before being stored in the database.

The following code demonstrates how bcrypt is used to securely hash user passwords during registration:

```
const hashedPassword = await bcrypt.hash(password, 10);

await db.query(sql` INSERT INTO users (full_name, email, username, password, role) VALUES
(${full_name}, ${email}, ${username}, ${hashedPassword}, 'user') `);
```

In this implementation, bcrypt is used to convert the original password into a secure hashed value before storing it in the database. Hashing is a one-way process, meaning the original password cannot be retrieved from the stored value. The number 10 represents the salt rounds, which determine how much computational work is required to generate the hash. Increasing this value improves security by making the hashing process slower and more resistant to attacks.

As shown in Figure 9 The password column now contains long, non-readable hashed values instead of plain text passwords, confirming that the mitigation has been successfully applied.

| id | full_name | email | username | password | role |
|---|---|---|---|---|---|
| 1 | Admin User | admin@example.com | admin | $2b$10$faKgmyK6URd0oIACsE1VZuhAeXzarh3i9r03oSs... | admin |
| 2 | Demo Althunayan | 444201003@student.ksu.edu.sa | demoTuna | $2b$10$1Cn0whxZ0zsmMeNs6gQkleAmQwIb4n1w/bb6... | user |
| 3 | Shmookh Almol... | shmookh@gmail.com | shmookh_almoliafai | $2b$10$a5dDoAQgEthQ08aNwBBy4eHsHuGKj5OWQftn... | user |
| 4 | Raghad Baselm | raghad@gmail.com | raghad_baselm | $2b$10$BZmi7s5l3zFbusDjDY/k2.4MHzoMPAC5Z3dxr.Lo... | user |
| 5 | Jood Alotaibi | Jood@gmail.com | jood_alotaibi | $2b$10$slN41.f2gLI7FiPIxQf1qut6bNMXihYVfTx1CmmJ4... | user |

*Figure 9: User passwords stored as bcrypt hashes instead of plain text after applying secure hashing*

This mitigation significantly improves the security of the application. Even if an attacker gains access to the database, the hashed passwords cannot be reversed to reveal the original values.

## C. CROSS-SITE SCRIPTING (XSS)

To address the XSS issues, two security improvements were implemented. The first improvement sanitizes user comments and removes any potentially malicious scripts before storing them. And the second is encoding the special character if found in the comments after fetching them.

The following code shows the sanitizing function with included comments that explain the steps of the text cleaning.

```
function sanitizeInput(input) {
    // Step 1: Decode HTML entities first to catch encoded attacks
    // Example: converts &#x3C; back to < before we sanitize
    let clean = input
        .replace(/&lt;/gi, '<')
        .replace(/&gt;/gi, '>')
        .replace(/&amp;/gi, '&')
        .replace(/&#x3C;/gi, '<')
        .replace(/&#x3E;/gi, '>')
        .replace(/&#(\d+);/gi, function (match, dec) {
            return String.fromCharCode(dec);
        })
        .replace(/&#x([0-9a-f]+);/gi, function (match, hex) {
            return String.fromCharCode(parseInt(hex, 16));
        });
    // Step 2: Remove all HTML tags
    clean = clean.replace(/<[\s\S]*?>/gi, '');
    // Step 3: Remove event handlers even without quotes and across line breaks
    clean = clean.replace(/on\w+[\s\S]*?=[\s\S]*?(['"]?[^'"]*['"]?)/gi, '');
    // Step 4: Remove javascript: and data: URIs
    clean = clean.replace(/javascript\s*:/gi, '');
    clean = clean.replace(/data\s*:/gi, '');
    // Step 5: Remove any leftover < > characters
    clean = clean.replace(/[<>]/g, '');
    // Step 6: Trim extra whitespace left behind
    clean = clean.trim();
    return clean;
}
```

And the following code shows how the function is implemented

```
        const { comment } = req.body;
        const username = req.session.user.username;
        const cleanComment = sanitizeInput(comment);
        await db.query(sql`
            INSERT INTO comments (username, comment)
            VALUES (${username}, ${cleanComment})
        `);
```

And since the function replaces the encoded characters /&#x3C to their original character, these special characters need to be passed as encoded ones after fetching to prevent any possible misinterpretation of these characters as HTML code, the following code :

```
        function escapeHTML(str) {
            return str
                .replace(/&/g, '&amp;')
                .replace(/</g, '&lt;')
                .replace(/>/g, '&gt;')
                .replace(/"/g, '&quot;')
                .replace(/'/g, '&#039;');
        }
```

```
comments.forEach(function (row) {
     html += '<table class="info-table" style="margin-bottom:10px;"><tbody>';
     html += '<tr><td style="width:120px;"><strong>' + escapeHTML(row.username) + '</strong></td>';
     html += '<td>' + escapeHTML(row.comment) + '</td>';
     html += '<td><button class="btn btn-danger" onclick="deleteComment(' + row.id +
')">Delete</button></td>';
     html += '</tr></tbody></table>';
   });
```

The following figures show how the application handles the malicious code:

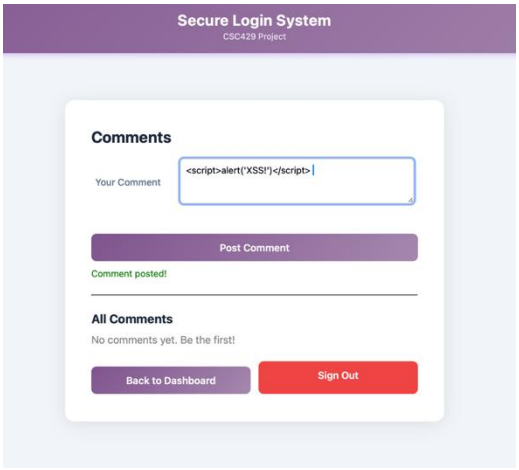



*Figure 10: The comment section before inserting the XSS attack comment.*

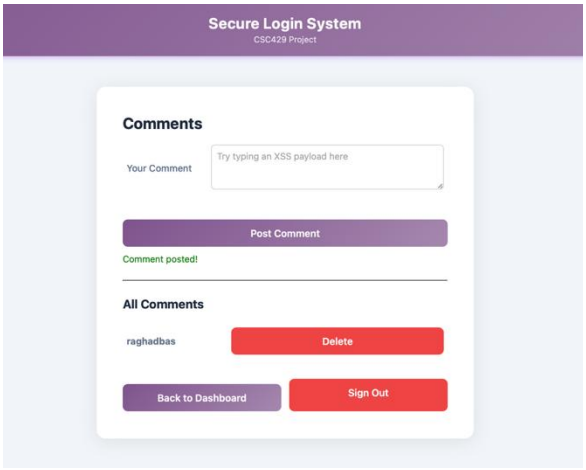



*Figure 11: The comment section after storing and retrieving the XSS attack comment.*

Altogether, these solutions provide mitigation of cross-site scripting by sanitizing input and encoding characters that can cause any possible misinterpretation in the HTML rendering process.

## D. ACCESS CONTROL

To address the access control vulnerability, a role-based access control (RBAC) mechanism was implemented to properly restrict access to sensitive routes within the application, especially the admin page (/admin). In the initial implementation, the system only verified that the user was authenticated, without checking the user's role, allowing any logged-in user to access the admin panel by directly entering the URL.
To fix this issue, a middleware function was introduced to enforce authorization checks based on the user's role stored in the session. After a successful login, the user's information, including their role, is saved in the session. This allows the application to validate permission for each incoming request.

The following code demonstrates the implemented access control mechanism:

```
function requireAdmin(req, res, next) {
    if (req.session && req.session.user && req.session.user.role === 'admin') {
        next(); // allow access
    } else {
        res.status(403).sendFile(path.join(__dirname, 'views', 'access-denied.html'));
    }
}

app.get('/admin', requireLogin, requireAdmin, (req, res) => {
    res.sendFile(path.join(__dirname, 'views', 'admin.html'));
});
```

In this implementation, the system first checks whether the user is logged in using requireLogin, then verifies whether the user has the required role using requireAdmin. If the user does not have administrative privileges, access to the admin page is denied, and an "Access Denied" page is displayed. This ensures that unauthorized users cannot access restricted functionality, even if they manually enter the URL.

As shown in Figure 12 When a normal user attempts to access the admin page after applying access control, the system correctly blocks the request and displays an "Access Denied" message instead of loading the admin dashboard.

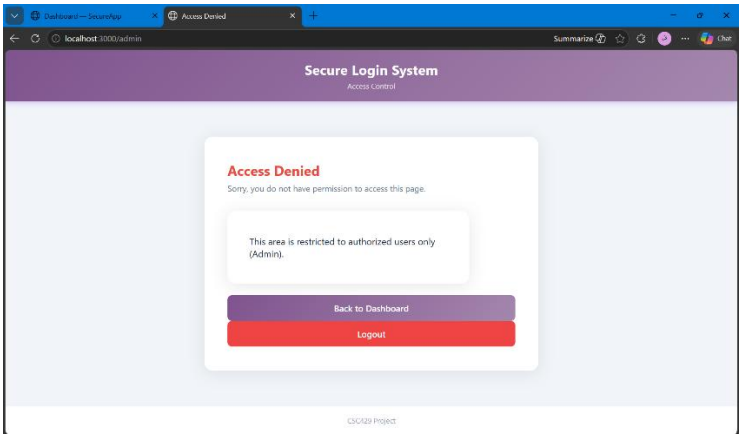



*Figure 12: Access denied when a non-admin user attempts to access the admin page*

Additionally, the same access control mechanism is applied to sensitive API routes such as retrieving user data and deleting users. These routes are protected using both requireLogin and requireAdmin, ensuring that only authorized users can perform administrative operations.

Overall, this mitigation effectively enforces proper authorization rules, preventing unauthorized access and significantly improving the security of the application.

## E. LACK OF ENCRYPTION

To address the encryption issues, two security improvements were implemented. First, HTTPS was enabled by configuring the server to use a self-signed SSL/TLS certificate. Figure 13 shows the URL using the HTTPS protocol. This ensures that all communication between the client and server is encrypted, protecting sensitive data such as login credentials and session identifiers from being intercepted or tampered with during transmission. Although the data is encrypted, the browser labels the website as "Not secure" because the certificate is self-signed rather than being issued by a trusted Certificate Authority (CA). Self-signed certificates are unsuitable for real-world use because they don't require proof of domain ownership, meaning an attacker can easily create their own certificate for your domain and present it to the user, gaining access to the transmitted data and defeating the purpose of encryption. Figure 14 shows the certificate details as presented in the browser.

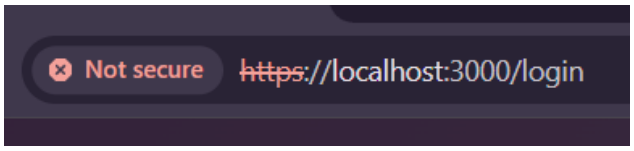



*Figure 13: The website now uses HTTPS instead of HTTP*

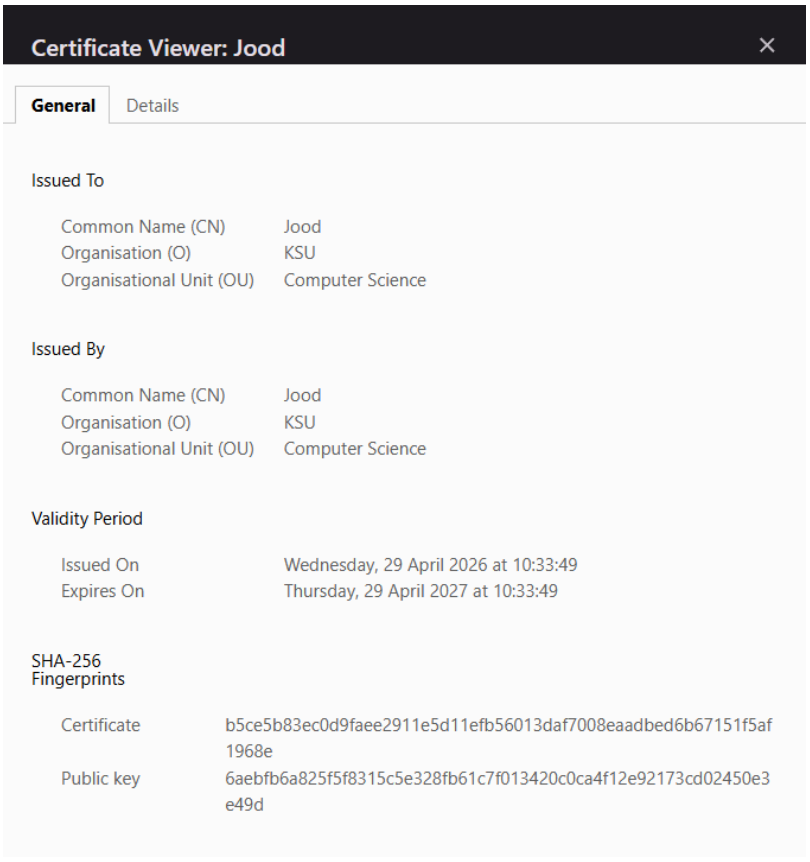



*Figure 14: Certificate details can be viewed in the browser*

The following snippet shows the code added to implement secure transmission over HTTPS:

```
const https = require('https');
const fs = require('fs');

https.createServer({
            key: fs.readFileSync('key.pem'),
```

```
            cert: fs.readFileSync('cert.pem')
        }, app).listen(PORT, () => {
            console.log(`Server running at : https://localhost:${PORT}`);
        });
```

Second, the session secret was removed from the source code and was instead stored in environment variables to enhance security. The application uses a .env file and the dotenv library to load the session secret at runtime, in addition to adding the .env file to .gitignore. This prevents sensitive information from being exposed in public repositories.

The following code shows how the server retrieves the session secret from the .env file:

```
app.use(session({
    secret: process.env.SESSION_SECRET, // from .env
    resave: false,
    saveUninitialized: false,
    cookie: {
        maxAge: 1000 * 60 * 60
    }
}));
```

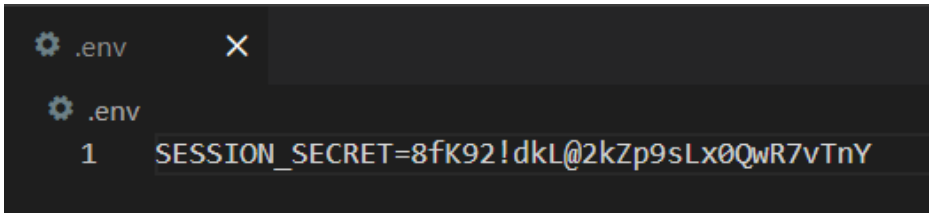



*Figure 15: Example of a .env file containing the session secret*

As shown in  Figure 15 The session secret is stored in the hidden .env file. These two measures ensure both confidentiality and integrity of transmitted data by encrypting it over HTTPS and securely handling session information to prevent forged session cookies.

# CHALLENGES

## A. SQL INJECTION

During the implementation of the SQL Injection vulnerability, an unexpected challenge was encountered. Initially, even when constructing the SQL query using direct string concatenation, the application did not exhibit SQL injection behavior as expected. As shown in Figure 16, when attempting to perform SQL injection using a malicious input (' OR 1=1 --), the application returned an error message stating “Something went wrong. Please try again.” instead of granting access.

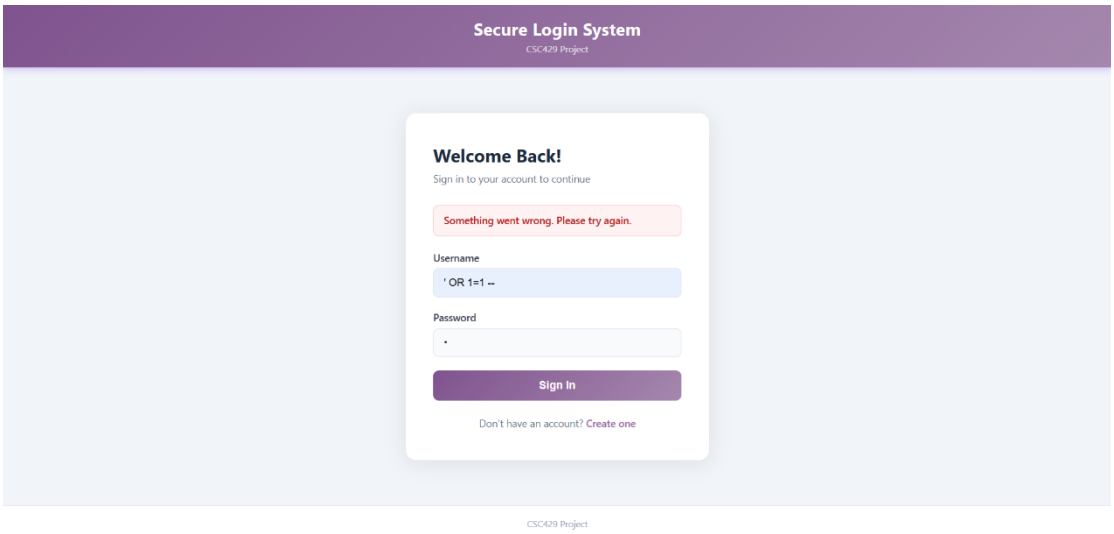



*Figure 16: Failed SQL Injection attempt due to built-in protection mechanisms*

This was due to built-in protection mechanisms provided by the database library, which automatically handled query parameterization and prevented unsafe execution of raw SQL input.

To intentionally demonstrate the vulnerability (as required by the project), it was necessary to bypass this protection using:

```
sql.__dangerous__rawValue(query)
```

This function forces the database to execute the raw SQL query without any sanitization or parameterization, thereby disabling the built-in security mechanisms.

While this allowed the vulnerability to be demonstrated, it also highlights the importance of modern frameworks and libraries in protecting applications against common attacks such as SQL injections.

## B. WEAK PASSWORD STORAGE

During the implementation of bcrypt password hashing, a challenge was encountered with the default admin account. While newly registered users had their passwords securely hashed before being stored, the admin account was automatically inserted during database initialization. As a result, its password was initially stored in plain text, as shown in Figure 17.

| | id | full_name | email | username | password | role |
|---|---|---|---|---|---|---|
| 1 | 1 | Admin User | admin@example.com | admin | admin123 | admin |
| 2 | 2 | Demo Althunayan | 444201003@student.ksu.edu.sa | demoTuna | $2b$10$Oamq63Y2RL8GhjwuUzLmc.qx9p.UkXFU... | user |
| 3 | 3 | Shmookh Almoli... | shmookh@gmail.com | shmookh_almoliafai | $2b$10$WYwMWgSBUIqF2TR34MQLFOwMEMzc... | user |
| 4 | 4 | Raghad Baselm | raghad@gmail.com | raghad_baselm | $2b$10$ZjMj5nIXlPSt0dT9btfAauGwIe/ajS6w.lgb.T... | user |
| 5 | 5 | Jood Alotaibi | Jood@gmail.com | jood_alotaibi | $2b$10$EiC4qaPSRdUlVGOBWysc/ura6u0PMJkpK... | user |

*Figure 17: Inconsistent password storage before the fix*

This caused inconsistency in the database, where regular user accounts were protected using bcrypt, while the admin account remained vulnerable. Since the admin is typically the first and most privileged account in the system, leaving its password unhashed posed a significant security risk.

To resolve this issue, the database initialization logic was modified to hash the admin password before inserting it into the users table. The following code shows how the fix was applied:

```
const adminHashedPassword = await bcrypt.hash('admin123', 10);
await db.query(sql` INSERT INTO users (full_name, email, username, password, role) VALUES ('Admin
User', 'admin@example.com', 'admin', ${adminHashedPassword}, 'admin') `);
```

This ensured that all accounts, including the default admin account, store passwords securely using bcrypt.

## C. CROSS-SITE SCRIPTING (XSS)

During the testing, this example <script>alert('XSS!')</script> did not trigger a pop-up even on the vulnerable page, then discovered that modern browsers block scripts injected via innerHTML for security reasons.

So to have more triggering use cases, switching to <img src=x onerror=alert('XSS!')>, which uses an image error event instead, successfully demonstrated the vulnerability.

## D. ACCESS CONTROL

During the implementation of access control, a challenge was encountered in ensuring that only admin users could access the admin page. Initially, the system only checked if the user was logged in, without checking the user's role, which allowed normal users to access the admin page.

After implementing role-based access control, the system correctly restricts access based on user roles. As shown in Figure 18 When a normal user attempts to access the admin page by entering the URL directly, the system denies access and displays an "Access Denied" message.

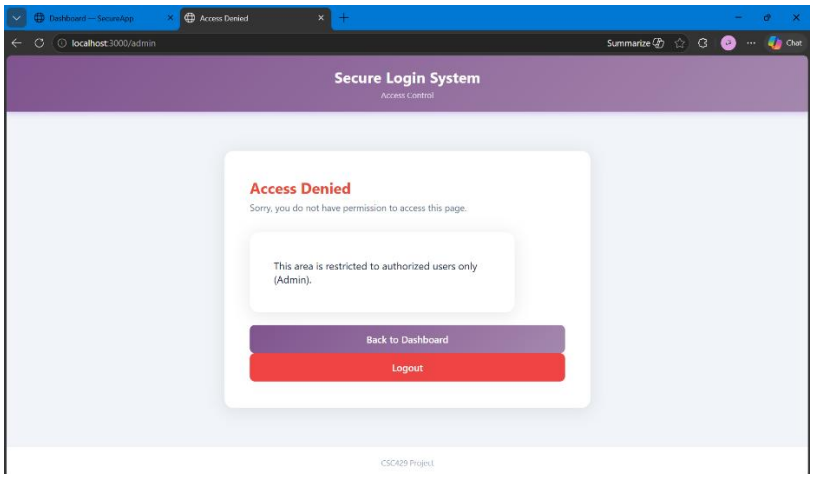



*Figure 18: Access denied when a normal user attempts to access the admin page*

To fix this issue, a middleware function (requireAdmin) was added to check the user's role before allowing access to the admin page. After applying this fix, only users with the admin role can access the /admin page, while normal users are denied access and shown an "Access Denied" message.

## E. LACK OF ENCRYPTION

Implementing encryption in the application presented some practical challenges related to setup and configuration. One of the main difficulties was setting up HTTPS in a local development environment. This required switching from the default HTTP setup to HTTPS and correctly loading the certificate and key files. Additionally, the same certificate and key had to be shared among group members to eliminate the need for each member to issue their own certificate.

Another challenge was related to certificate management. Getting a certificate signed by a trusted CA was not practical within the project's time constraints, as it would require registering and configuring a domain for validation. Therefore, a self-signed certificate was used for demonstration purposes.

Managing the session secret required careful handling. The .env file, used to securely store the secret, required exclusion from version control to ensure its contents are not leaked, as demonstrated in Figure 19. In addition, importing the correct library was crucial to allow the application to load environment variables at runtime.

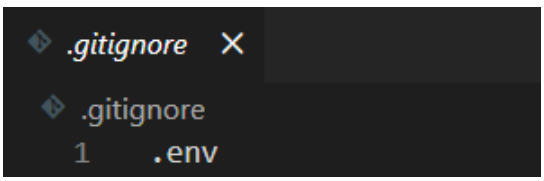



*Figure 19: Exposure of the session secret was prevented by adding .env to .gitignore*

# CONCLUSION

In conclusion, this project successfully demonstrated the process of identifying, analyzing, exploiting, and mitigating some of the most common and critical security vulnerabilities found in modern web applications. Throughout the development and testing phases, several vulnerabilities were examined in detail, including SQL Injection, weak password storage, Cross-Site Scripting (XSS), broken access control, and lack of encryption. Each vulnerability was carefully studied to understand its causes, potential impact, and the risks it could introduce to the confidentiality, integrity, and availability of the system.

The project also highlighted how insecure coding practices and insufficient security controls can expose web applications to serious threats such as unauthorized access, privilege escalation, session hijacking, credential theft, and sensitive data leakage. By intentionally demonstrating these vulnerabilities in a controlled environment, the project provided practical insight into how attackers exploit insecure systems and how easily security weaknesses can compromise an application if proper protections are not implemented.

To mitigate these risks, several security mechanisms and best practices were applied. Parameterized queries were implemented to prevent SQL Injection attacks, bcrypt hashing was used to securely store user passwords, input sanitization and output encoding were applied to mitigate XSS attacks, and role-based access control was introduced to properly restrict access to sensitive administrative functionality. In addition, HTTPS communication and secure session management techniques were implemented to improve the confidentiality and integrity of transmitted data and protect session information from unauthorized access.

Furthermore, the project provided valuable hands-on experience in secure web application development and demonstrated the importance of integrating security into every stage of the software development lifecycle. It also emphasized the critical role of modern security practices, secure coding standards, and proper authentication and authorization mechanisms in building reliable and secure systems.

Overall, this project successfully improved the security of the web application and demonstrated how applying appropriate mitigation techniques can significantly reduce common web application security risks. The project also reinforced the importance of cybersecurity awareness and secure development practices in protecting user data, maintaining system integrity, and defending against modern cyber threats.

# USE OF AI ASSISTANCE TOOLS

During the development and documentation of this project, AI-based tools such as ChatGPT and Claude were used for educational and learning purposes. These tools assisted in explaining cybersecurity concepts, clarifying programming-related questions, and improving understanding of vulnerabilities and mitigations.